



BUILD YOUR OWN

Community Events Platform

A friendly, soup-to-nuts guide for non-coders.

Stand up a full ticketing, venue, and community-events site on your own
— no Lovable required.

Originally created by the City of McKinney, Texas.

Released to the public under the GPL-3.0 open-source license.

Version 2 · 2026 edition

About this guide

This platform was originally built by the **City of McKinney, Texas** to power its community events, StreetBeats music program, venue and room reservations, sponsors, vendors, and ticket sales. The City has released the source code to the public under the **GPL-3.0 License** — a copyleft open-source license — so that any community, town, school district, non-profit, or organization can stand up the same software for themselves.

GPL-3.0 License — the short version

You can use, copy, modify, and distribute this software for free. If you build upon it and distribute your version, you must release your changes under the GPL-3.0 license too. The only requirement is that you keep the original attribution — "Created by the City of McKinney" — somewhere in the source. The software is provided "as is", with no warranty.

Who this guide is for

Anyone — even with little or no coding experience. We assume you know how to install programs on your computer and follow step-by-step instructions. Wherever a technical term shows up for the first time, we explain it in plain English.

What you'll have at the end

- A live website running on your own domain.
- A staff portal for managing events, venues, sponsors, vendors and musicians.
- A public-facing events page with ticket sales and QR-code check-in.
- An optional payment processor of your choice (USAePay, Stripe, Square, Authorize.Net, PayPal, etc.).
- A database that you own and control.

Contents

Part	Topic	Page
1	What you'll build	4
2	Tools you'll install	5
3	Get the source code	7
4	Set up the backend (database + auth)	8
5	Run the site on your computer	11
6	Customize the look and brand	12
7	Connect a payment processor	13
8	Using payment processors other than USAePay	15
9	Put it on the internet	18
10	Day-to-day operations	20
A	Troubleshooting	21
B	Glossary	22

PART 1

What you'll build

A complete community events platform: ticketing, venues, sponsors, musicians, vendors and a self-serve staff portal.

The major pieces

- **Public website** — events listings, individual event pages with ticket purchase, venue and room pages, musician profiles, and community organization signups.
- **Staff portal** — restricted area where staff manage events, approve sponsors and vendors, review applications, and check attendees in.
- **Database** — stores every event, ticket, application, and user account.
- **Authentication** — handles signups, logins, and password resets.
- **Payments** — accepts credit-card payments for paid tickets.

How the pieces talk to each other

Your visitors' browsers load the website. The website talks to a backend service (database + authentication + serverless functions). When a visitor buys a ticket, the backend calls your payment processor, records the transaction, and emails the ticket.

You own everything

Unlike a SaaS product, you control the source code, the database, the payment account, and the domain. You can change anything, host it anywhere, and never pay a per-user fee.

PART 2

Tools you'll install

Free, well-supported software. Install in this order and you're set.

Tool	Why you need it
Node.js (version 20 or newer)	The runtime that powers the website. Download the LTS installer from nodejs.org and click through the defaults.
Git	A tool for downloading and tracking source code. Install from git-scm.com . On Windows accept all defaults; on Mac/Linux it may already be installed.
Visual Studio Code (VS Code)	A free code editor from Microsoft. Download from code.visualstudio.com . You'll use it to open and lightly edit files.
A GitHub account	Free account at github.com . This is where your copy of the code will live.
A Supabase account (the backend)	Free account at supabase.com . This provides the database, authentication, and file storage all in one.
A Cloudflare account (the hosting)	Free account at cloudflare.com . This is where your live website will run.

How to confirm Node and Git are installed

Open a Terminal (Mac/Linux) or PowerShell (Windows) and run these two commands. You should see version numbers, not errors:

```
node --version
git --version
```

PART 3

Get the source code

Make your own copy of the original City of McKinney code on GitHub.

Step 1 — Fork the repository

- 1 Sign in to github.com.
- 2 Open the City of McKinney repository (your contact from the City can share the URL).
- 3 Click the **Fork** button at the top right. This creates your own copy.

Step 2 — Download your fork to your computer

Open a terminal and run (replace YOURNAME with your GitHub username):

```
git clone https://github.com/YOURNAME/community-events-platform.git
cd community-events-platform
```

Step 3 — Install the website's building blocks

```
npm install
```

This downloads the libraries the project depends on. It can take a few minutes the first time.

Keep the City of McKinney credit

The GPL-3.0 license is in the **LICENSE** file at the root of the project. Please leave that file in place — it's the only requirement for using the software freely.

PART 4

Set up the backend (database + auth)

Create a free Supabase project — this stores all your data.

Step 1 — Create a project

- 1 Sign in at supabase.com and click **New project**.
- 2 Give it a name (e.g. *Your-City Events*), choose a strong database password, and pick a region near you.
- 3 Wait 1–2 minutes for it to spin up.

Step 2 — Grab three values you'll need

In Supabase, open **Project Settings** → **API**. Copy and save these three values:

- **Project URL** — looks like <https://abcd1234.supabase.co>
- **anon public key** — safe to put in the website's code.
- **service_role key** — secret. Keep this in environment variables only.

Step 3 — Create the database tables

The project ships with a folder called **supabase-migrations/** containing numbered SQL files (001, 002, 003...). These build the tables.

- 1 In Supabase, open **SQL Editor** → **New query**.
- 2 Open each migration file in VS Code in order (001 first, then 002, then 003 ...).
- 3 Copy each file's contents, paste into the SQL Editor, and click **Run**.
- 4 If a query reports an error you don't understand, stop and consult the Troubleshooting appendix.

Step 4 — Turn on email signups

- 1 Open **Authentication** → **Providers** in Supabase.
- 2 Make sure **Email** is enabled.
- 3 (Optional) Enable **Google** or **Apple** if you want one-click sign-in.

Step 5 — Make yourself a staff user

- 1 Run the site once (next chapter) and create a normal account with your email.
- 2 Back in Supabase, open **Table Editor** → **user_roles** and add a row: your user id, role = *admin*.
- 3 Log out and back in. You now have access to */staff*.

PART 5

Run the site on your computer

Once everything's installed, you can preview the whole platform locally.

Step 1 — Create a settings file

In the project folder, make a new file called `.env`. Open it in VS Code and paste the following, replacing the placeholder values with the three values you copied from Supabase:

```
VITE_SUPABASE_URL=https://YOUR-PROJECT.supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY=your-anon-public-key
SUPABASE_URL=https://YOUR-PROJECT.supabase.co
SUPABASE_PUBLISHABLE_KEY=your-anon-public-key
SUPABASE_SERVICE_ROLE_KEY=your-service-role-key
```

Keep `.env` private

Never share this file or commit it to GitHub. It contains keys that can read and write your entire database.

Step 2 — Start the site

```
npm run dev
```

Open the address it prints (usually **`http://localhost:5173`**) in your browser. You should see the homepage.

Step 3 — Stop the site

In the terminal, press **Ctrl + C**.

PART 6

Customize the look and brand

Make the site yours — name, colors, logo and content.

Change the site name and logo

- 1 Open `src/components/site-header.tsx` in VS Code.
- 2 Find the text that reads the city name and replace it with your organization's name.
- 3 Drop your logo image into `src/assets/` and import it in the same file.

Change the colors

Open `src/styles.css`. Near the top is a block of color variables (written in the *oklch* color format). Change the `--primary`, `--accent`, and `--background` values to match your brand. The whole site picks up the new colors automatically.

Manage content from the Staff Portal

Most content — venues, rooms, stages, sponsors, vendors, events, applications — is managed from the **Staff** area inside the running site. You don't need to edit code for day-to-day updates.

Turn modules on or off

If your community doesn't need (for example) the StreetBeats music program, you can disable it from **Staff/admin** under *Platform Modules*. Disabled modules hide their menu items and signup pages.

PART 7

Connect a payment processor

Default flow uses USAePay. The next chapter shows how to swap in others.

Step 1 — Sign up with USAePay

- 1 Go to usaepay.com and apply for a merchant account.
- 2 Once approved, log in to the USAePay console.
- 3 Open **Settings** → **API Keys** and create a new key. Save the **Key** and **PIN**.

Step 2 — Add the keys to your environment

Add these three lines to your `.env` file (locally) and to your hosting environment variables (in production):

```
USAEPAY_API_KEY=your-api-key  
USAEPAY_API_PIN=your-api-pin  
USAEPAY_MODE=sandbox
```

Use **sandbox** while testing. When you're ready to take real money, change it to **live** and redeploy.

Step 3 — Test a purchase

- 1 Restart the site (`npm run dev`).
- 2 Create a test event with a paid ticket tier.
- 3 Buy a ticket using a USAePay test card (the docs list these).
- 4 Confirm the ticket appears under `/my-tickets`.

Where the payment code lives

All payment logic is in `src/lib/payments.functions.ts`. That is the single file you adapt when switching to another processor.

PART 8

Using a payment processor other than USAePay

The platform is processor-agnostic. Here's the recipe for swapping in any credit-card processor that has an HTTPS API.

The general pattern

Every processor works roughly the same way: you send the card details (or a token) to their API along with the amount, and they reply with either an approval or a decline. The file **src/lib/payments.functions.ts** is where that conversation happens. You replace the USAePay-specific block with calls to your chosen processor.

Generic step-by-step (any processor)

- 1 Open an account with the processor and get your API credentials.
- 2 Read their API docs for “**create charge**” or “**create payment**”.
- 3 Add the credentials as new environment variables (never hard-code them).
- 4 Open **src/lib/payments.functions.ts** and replace the *fetch()* call to USAePay with one to your processor.
- 5 Translate their response into the existing fields: *approved*, *transaction_ref*, *auth_code*, *error_message*.
- 6 Test in their sandbox/test mode. Only flip to live keys after a successful end-to-end test.

Popular processors and what to know

Processor	Best for	Key environment variables	Notes
Stripe	Almost any use case; best developer experience.	STRIPE_SECRET_KEY, STRIPE_PUBLISHABLE_KEY, STRIPE_WEBHOOK_SECRET	Use Stripe Checkout (hosted page) for the easiest PCI compliance — redirect the buyer to Stripe, then back to a success page.
Square	Organizations that already have Square for in-person sales.	SQUARE_ACCESS_TOKEN, SQUARE_LOCATION_ID, SQUARE_ENVIRONMENT	Use the Square Web Payments SDK to tokenize the card on the browser, then call POST /v2/payments from the server.

Processor	Best for	Key environment variables	Notes
Authorize. Net	Government/municipal contracts that require it.	AUTHNET_LOGIN_ID, AUTHNET_TRANSACTION_KEY, AUTHNET_ENV	Use the Accept.js library client-side to get a payment nonce, then call <code>createTransactionRequest</code> from the server.
PayPal / Braintree	Sites that want PayPal and Venmo buttons.	PAYPAL_CLIENT_ID, PAYPAL_CLIENT_SECRET, PAYPAL_ENV	Use PayPal Orders v2 API. Render the PayPal Buttons SDK on the ticket page; capture the order from the server.
Adyen	International / multi-currency events.	ADYEN_API_KEY, ADYEN_MERCHANT_ACCOUNT, ADYEN_ENV	Use Adyen Web Drop-in client-side; call <code>/payments</code> from the server.

Example: replacing USAePay with Stripe Checkout

Stripe Checkout is the simplest swap because Stripe hosts the actual card-collection page for you. The change in `src/lib/payments.functions.ts` becomes (simplified):

```
// 1. Install once: npm install stripe
import Stripe from 'stripe';
const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!);

// 2. Inside the handler, replace the USAePay fetch with:
const session = await stripe.checkout.sessions.create({
  mode: 'payment',
  line_items: [{
    price_data: {
      currency: 'usd',
      product_data: { name: tier.name },
      unit_amount: amountCents,
    },
    quantity,
  }],
  success_url: `${origin}/events/${data.session_id}?paid=1`,
  cancel_url: `${origin}/events/${data.session_id}?cancelled=1`,
  customer_email: data.email,
});
return { checkout_url: session.url };
```

Then on the frontend, instead of submitting card fields, redirect the buyer to `checkout_url`. Create the attendee row inside a Stripe webhook (`checkout.session.completed`) so you only register paid tickets.

PCI compliance, in plain English

If you let the card number touch your own server (USAePay direct, Authorize.Net AIM), your hosting must meet PCI-DSS rules. If you use a **hosted checkout** (Stripe Checkout, PayPal, Square hosted) or a **browser-side tokenizer** (Stripe Elements, Square Web Payments, Authorize.Net Accept.js), the card data never reaches your server and your compliance scope shrinks dramatically. Pick a hosted/tokenized option whenever you can.

PART 9

Put it on the internet

Deploy to Cloudflare Pages and point your domain at it.

Step 1 — Push your code to GitHub

In the terminal:

```
git add .  
git commit -m "My customizations"  
git push origin main
```

Step 2 — Connect to Cloudflare Pages

- 1 Sign in to cloudflare.com and open **Workers & Pages**.
- 2 Click **Create application** → **Pages** → **Connect to Git**.
- 3 Pick the GitHub repository you just pushed.
- 4 Build command: **npm run build**. Output directory: **dist**.
- 5 Click **Save and Deploy**. First build takes 2–5 minutes.

Step 3 — Add environment variables

In the Cloudflare Pages project, open **Settings** → **Environment variables** and add the same variables from your **.env** file (Supabase URL, anon key, service key, payment processor keys). Redeploy.

Step 4 — Point your domain at it

- 1 In Cloudflare Pages, open **Custom domains** → **Set up a custom domain**.
- 2 Enter your domain (e.g. events.yourcity.gov).
- 3 Cloudflare walks you through adding a CNAME or NS record at your registrar.
- 4 Within an hour, your domain serves the site over HTTPS automatically.

PART 10

Day-to-day operations

What you'll do once the site is live.

Selling tickets and checking attendees in

- Create events from **/staff/events**.
- Attendees buy tickets from the public event page.
- At the door, open **/staff/attendees** on a phone and scan the QR code on each ticket.

Refunds

Refunds are issued directly inside your payment processor's dashboard (USAePay console, Stripe dashboard, etc.). After refunding, you can mark the attendee as cancelled in **/staff/attendees**.

Backups

Supabase makes daily automatic backups on paid plans. On the free tier, open **Database → Backups** and download a manual backup at least once a week.

Updating the software later

```
git pull origin main
npm install
npm run build
git push
```

Cloudflare detects the push and redeploys automatically.

PART A

Troubleshooting

Common problems and the fix.

Symptom	Likely fix
"command not found: npm"	Node.js isn't installed (or your terminal needs to be reopened). Install Node.js LTS from nodejs.org and open a fresh terminal.
"Invalid login credentials" when signing in	The email/password combo is wrong, or the user wasn't confirmed. In Supabase → Authentication → Users, confirm the user manually.
Payments page says "Payments are not yet configured"	Your environment variables are missing or misspelled. Double-check <code>USAEPAY_API_KEY</code> and <code>USAEPAY_API_PIN</code> locally and in Cloudflare.
Staff portal says "No access"	You haven't been given the <i>admin</i> or <i>staff</i> role yet. In Supabase, open the user_roles table and insert a row for your user.
Site builds locally but fails on Cloudflare	Almost always a missing environment variable in Cloudflare Pages. Mirror your <code>.env</code> exactly.

PART B

Glossary

Plain-English definitions for the jargon used in this guide.

Term	Meaning
Repository ("repo")	A folder of source code, typically stored on GitHub.
Fork	Your own personal copy of someone else's GitHub repository.
Clone	Downloading a repository onto your computer.
Environment variable	A configuration value (like an API key) kept outside the code.
Migration	A SQL file that creates or updates database tables.
Sandbox / test mode	A safe environment in a payment processor that uses fake cards.
Webhook	An automatic notification sent by an external service to your site.
Tokenization	Replacing a card number with a one-time code so it never touches your server.
RLS (Row Level Security)	Database rules that decide which user can read or write which row.
GPL-3.0 License	A copyleft open-source license — free to use and modify, with the requirement that distributed derivative works also use GPL-3.0.

This guide and the underlying software are an open-source contribution from the **City of McKinney, Texas**, released under the **GPL-3.0 License**. You are free to use, modify, and distribute it, including for commercial use, provided the original attribution notice remains in the source.